# TEST PLAN DOCUMENT

## AutoSupply POS and Inventory Management System (Web-Based)

## 1 Introduction

The Test Plan is designed to prescribe the scope, approach, resources, and schedule of all testing activities of the project AutoSupply POS and Inventory Management System.

The plan identifies the items to be tested, the features to be tested, the types of testing to be performed, the personnel responsible for testing, the resources and schedule required to complete testing, and the risks associated with the plan.

### 1.1 Scope

#### 1.1.1 In Scope

All the features of the AutoSupply POS and Inventory Management System which were defined in the software requirement specification are needed to be tested:

| No. | Module Name | Applicable Roles | Description |
|---|---|---|---|
| 1 | User Authentication | Admin, Cashier | Sign in/out using username or email, session handling, and password validation; new user can register an account (name, username, email, password); user can ask for a password-reset link by email; account will lock for a short time if login is wrong many times; session will end automatically if the user is not active; user can view and update their own account details and password; password is saved using secure hashing, not plain text |
| 2 | Dashboard | Admin, Cashier | Today's revenue and order count, top-selling product, low-stock alert count, total catalog (SKU) count, 7-day daily sales trend chart, revenue-share chart, top 5 best-selling parts chart, and a low-stock warnings list with a quick restock action |
| 3 | Inventory Catalog | Admin, Cashier | Add, update, delete, search, and restock parts (product name, additional name/specs, category, brand, price, quantity, shelf location, description); filter by category and by low-stock status; the product list has pages and shows the total number of products, low-stock items, and out-of-stock items |
| 4 | POS Terminal | Admin, Cashier | Search/filter parts, build an active ticket, adjust order item quantities, accept customer payment (manual entry, quick-add buttons, or exact amount), compute change, record the sale, generate the invoice, and print the receipt |
| 5 | Sales Records & Invoices | Admin, Cashier | View grouped (invoice-level) sales and individual product sales, filter by date range and search, and view/print individual receipts; can search by invoice number, product, or user; each invoice shows the staff/owner account who made the sale; the invoice list has pages |
| 6 | Stock Auto-Deduction | System | Automatic reduction of product quantity after a recorded sale; this is done in one single step so the stock will |

| | | | |
|---|---|---|---|
| | | | never go below zero, even if two sales happen at almost the same time |
| 7 | Audit Trail | Admin, Cashier | Logging of product inserts/updates, restocks, and POS-sale quantity deductions, with old-value vs. new-value tracking, for accountability; the audit trail page also has pages (pagination) |
| 8 | Backend Services | System | Controllers/routes handling checkout and recording of sales, inventory CRUD and restock, dashboard/report queries, and audit logging |
| 9 | Data Integrity & Business Rules | System | Stock will not go below zero when a sale is recorded, the data connecting users, products, invoices, and sales are kept correct, and the money amounts are computed consistently without rounding errors |

### 1.1.2 Out of Scope

These features are not to be tested because they are not included in the software requirement specification:

- Third-party payment gateway integration (system records cash payment/change only)
- Multi-branch/multi-store synchronization
- Native mobile application version
- Hardware interfaces (barcode scanners, receipt printers) unless explicitly added to requirements
- Server infrastructure security hardening and penetration testing
- Website performance/load testing beyond normal functional use
- Product image upload/display (this is only for future use; it is not part of the requirements yet)
- Many users using the system at the same time on different devices (the system is made for only one computer and one user at a time)
- HTTPS/TLS security between the browser and the server (the system only runs on the local computer; the computer's own user account and file permissions will protect it instead)
- Sending password-reset emails, unless an email-sending service is set up (if it is not set up, the reset request will just be saved in the log instead of being emailed)

### 1.2 Quality Objective

To verify the functionality of the AutoSupply POS and Inventory Management System, the project will focus on testing the inventory, transaction, and reporting modules, to guarantee all these operations can work normally in a real business environment.

### 1.3 Roles and Responsibilities

The project will use the assigned student developers and testers within the team to conduct testing.

| No. | Member | Tasks |
|---|---|---|
| 1 | **Jared Jeffrey Abellera — Team Lead** | Manage the whole QA effort and define testing priorities<br>Acquire resources needed for testing (time, environment, personnel)<br>Give final sign-off before submission, based on the exit criteria in 2.4 |
| 2 | **Mica Danah Paris — Developer** | Set up and maintain the test environments (SQLite dev / Dockerized MySQL QA)<br>Implement automated backend test scripts (PHPUnit feature tests) alongside test cases designed by QA<br>Fix defects reported by QA and resolve issues found during regression testing |
| 3 | **Gemwil Alcaraz — Quality Assurance** | Identify appropriate test techniques and design test cases<br>Execute tests, log results, and report defects with steps to reproduce |
| 4 | **Roel Bendaña — Quality Assurance** | Execute tests, log results, and report defects with steps to reproduce<br>Support regression testing after defect fixes or feature changes |
| 5 | **Sofia La-Arni Tapongco — System Analyst** | Verify that test cases trace back to the FRS/NFRS requirements<br>Confirm the testing process is meeting the specified requirements |
| 6 | **Theresa Valiente — Librarian (Documentation)** | Maintain and version the test plan, test case documents, and test design specifications |
| 7 | **Erich Castillo — Librarian (Documentation)** | Maintain test result reports, defect reports, and release notes |

## 2 Test Methodology

### 2.1 Overview

This section describes the levels of testing, defect handling process, and criteria used to determine whether the AutoSupply POS and Inventory Management System is ready for release.

### 2.2 Test Levels

For the AutoSupply POS and Inventory Management System, the following levels of testing will be conducted:

- Unit Testing – individual backend functions (e.g., stock deduction, total computation) tested in isolation
- Integration Testing – frontend, backend, and database modules combined and tested as a group
- System Testing – the complete, integrated web application evaluated against the specified requirements
- Backend/Controller Testing – Laravel controllers and business logic (checkout, inventory CRUD, restock, audit logging) tested via PHPUnit feature tests

- Smoke Testing – basic sanity checks after each build/deployment
- Regression Testing – re-testing affected modules after defect fixes or feature changes
- User Acceptance Testing (UAT) – validation with the adviser and/or end-user representative before final submission
- Usability Testing – checking if the layout looks good and consistent on desktop and tablet screens, and if common tasks (like recording a sale or adding a product) can be done in just a few steps
- Performance Testing – checking if pages and the dashboard load in 3 seconds or less under normal use, and if the product, invoice, and audit-trail lists still load fast even with many records, using pagination
- Security Testing – checking if the password is hashed safely, if user input is checked/cleaned to prevent SQL injection and XSS, if the session ends after being inactive, and if login locks after many wrong tries
- Reliability & Data Integrity Testing – checking if stock deduction works correctly and never goes negative, if the data connecting users, products, invoices, and sales stay correct, and if money amounts are computed consistently
- Compatibility Testing – checking if the system works correctly on the latest Chrome, Edge, and Firefox browsers when opened locally (example: http://localhost)

### 2.3 Bug Triage

Reported defects are logged with steps to reproduce, severity, and module affected. The team reviews and prioritizes defects before assigning them for a fix, following this severity classification: Severity 1 – critical/blocking, Severity 2 – major, Severity 3 – minor, Severity 4 – cosmetic.

### 2.4 Acceptance Criteria

#### 2.4.1 Entry-Exit Criteria

- Entry Criteria for the start of testing will be a hand-off from the developers
- Exit Criteria for the end of testing would be:
    - No open Severity 1 or Severity 2 defects
    - 100% test completion
    - At least 95% test pass rate

#### 2.4.2 Suspension Criteria and Resumption Requirements

If the team members report that 40% or more of the executed test cases failed, testing will be suspended until the development team fixes all the failed cases. Testing resumes once a fixed build is re-deployed to the test environment.

### 2.5 Test Completion / Sign-off

- Specifies the criteria that denote a successful completion of a test phase
- Mandatory run rate of 100% unless there is a clear, documented reason to waive a test case
- Pass rate of 95% is mandatory; achieving the pass rate is required for sign-off

### 2.6 Project Task, Estimation, and Schedule

| Task | Members | Planned Dates | Estimated Effort (man-hour) |
|---|---|---|---|
| **Create the test specification** | Sofia La-Arni Tapongco (System Analyst); Gemwil Alcaraz, Roel Bendaña (QA) | Oct 21–22, 2026 | ~48 (est.) |
| **Perform test execution** | Gemwil Alcaraz, Roel Bendaña (QA); Mica Danah Paris (Developer, environment support) | Oct 23–26, 2026 | ~48 (est.) |
| **Bug Fixing & Retesting** | Mica Danah Paris (Developer); Gemwil Alcaraz, Roel Bendaña (QA) | Oct 27–28, 2026 | ~48 (est.) |
| **Deployment Testing** | Mica Danah Paris (Developer); Gemwil Alcaraz, Roel Bendaña (QA) | Oct 29, 2026 | ~24 (est.) |
| **Test report** | Gemwil Alcaraz, Roel Bendaña (QA); Erich Castillo (Documentation) | Oct 26 & Nov 3–4, 2026 | ~48 (est.) |
| **Test delivery** | Jared Jeffrey Abellera (Team Lead) | Nov 4–5, 2026 | ~16 (est.) |
| **Total** | | | ~232 (est.) |

Dates above follow the Software Quality Stage of the project timeline (Test Planning through Deployment Testing, Oct 21–Nov 5, 2026). Effort estimates assume an 8-hour day per assigned member (duration × 8 × number of members on that task); these are rough estimates for planning purposes and should be revised once actual task-level effort is tracked.

## 3 Test Deliverables

**Before the Testing Phase**

- Test plan document (this document)
- Test case documents
- Test design specifications

**During Testing**

- Testing tool: PHPUnit (Laravel's built-in testing framework, run via "php artisan test")
- Test data – to be provided by the assigned developer(s)
- Test Traceability matrix / Defect-Test monitoring tool for logging errors and execution progress

**After the Testing Cycles**

- Test results/reports
- Defect report completion
- Installation/deployment guide for the web application
- Release notes

## 4 Resource & Environment Needs

### 4.1 Testing Tools

| No. | Resources | Descriptions |
|---|---|---|
| 1 | Server | Local application server (php artisan serve or Laravel Sail) or Apache/XAMPP hosting the Laravel application; SQLite (default) or MySQL/MariaDB database. Based on the system design, there are two setups: a Development setup that uses a local SQLite database file, and a Quality-Assurance (QA) setup where the Laravel app and MySQL database run in their own Docker containers, managed using Docker Compose |
| 2 | Test Tool | PHPUnit (php artisan test) for backend/controller testing; browser developer tools for frontend inspection; Laravel Artisan Tinker for ad hoc data checks |
| 3 | Network | Stable LAN/internet connection for accessing the web application during testing. The system only runs on the local computer, so internet is not needed to use it; internet is only needed if testing the password-reset email feature, and only after an email service is already set up |
| 4 | Computers | Devices with modern web browsers for cross-browser testing |
| 5 | Containerization | Docker and Docker Compose, used to run the QA environment (the Laravel app container, MySQL container, and a phpMyAdmin container to view the database directly). This can be started with just one build/run command |

### 4.2 Test Environment

For backend testing, the required tool is PHPUnit, run via "php artisan test". Test files are organized under tests/Feature and tests/Unit and are to be shared with the team via the group repository for review.

Based on the system design, there are two environment setups: a Development setup that uses a local SQLite database file (no Docker container needed), and a Quality-Assurance (QA) setup where the Laravel app and a MySQL database each run inside their own Docker container, managed using Docker Compose, together with a phpMyAdmin container to view the database directly. To switch between these two setups, only the environment settings need to change, not the code itself.

The following are required for testing the web-based system, in addition to the test machine's base software:

- A modern web browser (Chrome, Edge, or Firefox – latest version)
- SQLite (default local/testing database) or MySQL/MariaDB (via XAMPP). For the QA environment, MySQL 8.4 running inside Docker is used instead of MySQL from XAMPP, based on the system design
- PHP 8.3, Composer, and Node.js/npm (for the Vite-based frontend build)
- Windows 10/11 or an equivalent OS capable of running the above
- Docker Desktop (or Docker Engine) and Docker Compose, needed for the Docker-based MySQL QA environment